Module 6 — Spring Security

> Spring Security is a chain of servlet filters. The must-know answer is the **complete authentication flow and the Security Filter Chain**. Interviewers also drill JWT vs sessions, OAuth2, and CSRF/CORS.

6.1 Core Concepts: Authentication vs Authorization

- **Authentication (AuthN)** — *who are you?* Verify identity (credentials, token).
- **Authorization (AuthZ)** — *what can you do?* Check permissions/roles.
- Key abstractions: `Authentication` (principal + authorities), `SecurityContext` (holds the current `Authentication`, stored in `SecurityContextHolder`, ThreadLocal by default), `UserDetails` / `UserDetailsService`, `AuthenticationManager` / `AuthenticationProvider`, `GrantedAuthority`.

6.2 The Security Filter Chain (the classic question)

Why Interviewers Ask This

Security “magic” is just an ordered filter chain. Naming the key filters proves real understanding.

Core Concept

A `FilterChainProxy` (`springSecurityFilterChain`) delegates to an ordered list of `SecurityFilter`s. Each request passes through them before reaching the DispatcherServlet.

Key filters (order matters)

```
Request
  │
  ▼
SecurityContextPersistenceFilter / SecurityContextHolderFilter
  │   (load SecurityContext from session/repo)
  ▼
CorsFilter -> CsrfFilter
  ▼
(Authentication filters)
  UsernamePasswordAuthenticationFilter   (form login /login POST)
  BearerTokenAuthenticationFilter        (OAuth2 resource server / JWT)
  BasicAuthenticationFilter              (HTTP Basic)
  │
  ▼
ExceptionTranslationFilter   (catches AuthN/AuthZ exceptions ->
                              401 entry point / 403)
  │
  ▼
AuthorizationFilter (was FilterSecurityInterceptor)  (authorize request)
  │
  ▼
DispatcherServlet -> Controller
```

Authentication Flow (username/password)

```
1. UsernamePasswordAuthenticationFilter builds an unauthenticated
   UsernamePasswordAuthenticationToken(username, password)
2. -> AuthenticationManager (ProviderManager)
3. -> AuthenticationProvider (DaoAuthenticationProvider)
4.     -> UserDetailsService.loadUserByUsername()
5.     -> PasswordEncoder.matches(raw, stored)
6. success: build authenticated Authentication (with authorities)
7. store in SecurityContextHolder (and session, if stateful)
8. on failure: AuthenticationException -> 401 via entry point
```

ASCII — AuthN Components

```
 filter -> AuthenticationManager (ProviderManager)
             -> [ DaoAuthenticationProvider ] -> UserDetailsService + PasswordEncoder
             -> [ JwtAuthenticationProvider ] -> validate signature/claims
          returns authenticated Authentication -> SecurityContext
```

Real Production Example (modern lambda DSL, Spring Security 6)

```
@Bean
SecurityFilterChain chain(HttpSecurity http) throws Exception {
  http.csrf(csrf -> csrf.disable())                // stateless API
      .cors(Customizer.withDefaults())
      .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
      .authorizeHttpRequests(a -> a
          .requestMatchers("/public/**", "/actuator/health").permitAll()
          .requestMatchers("/admin/**").hasRole("ADMIN")
          .anyRequest().authenticated())
      .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()));
  return http.build();
}
```

Interview Q / Follow-ups

- Walk through the security filter chain / authentication flow.
- `AuthenticationManager` vs `AuthenticationProvider` vs `UserDetailsService`.
- Where is the authenticated user stored? *(SecurityContextHolder → ThreadLocal.)*
- What does `ExceptionTranslationFilter` do? *(AuthN → 401 entry point; AuthZ → 403.)*
- How to add a custom filter? *(`addFilterBefore/After`.)*

Hands-on Exercise

Add a custom `OncePerRequestFilter` that reads an API key header and populates the `SecurityContext` before `AuthorizationFilter`.

6.3 Password Encoding

Store **hashed + salted** passwords. Use `BCryptPasswordEncoder` (adaptive, salted) or `Argon2` / `PBKDF2`. `DelegatingPasswordEncoder` (default) prefixes the algorithm `{bcrypt}...` enabling migration. **Never** store plaintext or use fast/unsalted hashes (MD5/SHA-1).

Interview Q

Why BCrypt over SHA-256? *(slow, salted, adaptive work factor resists brute force.)* How to rotate encoders? *(DelegatingPasswordEncoder.)*

6.4 JWT, Sessions, OAuth2 & OpenID Connect

Session vs JWT

	Session (stateful)	JWT (stateless)
Server state	session store (memory/Redis)	none (token self-contained)
Scaling	needs shared session store / sticky	scales horizontally easily
Revocation	easy (delete session)	hard (until expiry) → short TTL + refresh + denylist
Payload	opaque id	signed claims (sub, roles, exp)
Use	classic web apps	APIs, microservices, SPAs/mobile

JWT structure

`header.payload.signature` (Base64URL). Signature (HMAC or RSA/EC) proves integrity; **payload is not encrypted** — don't put secrets in it. Validate: signature, `exp`, `iss`, `aud`.

OAuth2 roles & flow

- Roles: **Resource Owner** (user), **Client** (app), **Authorization Server** (issues tokens), **Resource Server** (your API).
- **Authorization Code + PKCE** is the recommended flow for web/mobile/SPA.
- **Client Credentials** for service-to-service.
- **OpenID Connect (OIDC)** layers *authentication* on OAuth2 by adding an **ID token** (a JWT with user identity claims). OAuth2 alone = authorization; OIDC adds login/identity.

ASCII — Authorization Code + PKCE

```
User -> Client: login
Client -> AuthServer: /authorize (code_challenge)
User authenticates at AuthServer
AuthServer -> Client: authorization code (redirect)
Client -> AuthServer: /token (code + code_verifier)
AuthServer -> Client: access token (+ id token, refresh)
Client -> Resource Server: request + Bearer access token
Resource Server: validate JWT (jwks) -> serve
```

Real Production Example

Microservices behind an API gateway: the gateway/authorization server issues JWTs; each service is an **OAuth2 resource server** validating the JWT signature against the JWKS endpoint (`spring.security.oauth2.resourceserver.jwt.jwk-set-uri`) and mapping claims → authorities. No shared session store needed.

Common Mistakes / Trade-offs

- Long-lived JWTs with no revocation strategy.
- Putting sensitive data in JWT payload (readable).
- `alg=none` / not validating signature.
- Storing JWT in `localStorage` (XSS risk) vs httpOnly cookie (CSRF considerations).

Interview Q / Follow-ups

- Session vs JWT — trade-offs, when each?
- How do you revoke a JWT? *(short TTL + refresh token rotation + denylist.)*
- Explain OAuth2 authorization code + PKCE; why PKCE?
- OAuth2 vs OIDC?
- Where to store tokens on the client?

6.5 Method Security

`@EnableMethodSecurity` → `@PreAuthorize("hasRole('ADMIN')")`, `@PostAuthorize`, `@PreFilter`/`@PostFilter`, SpEL with `#params` and `authentication`. Enforces AuthZ at the service layer, not just URLs.

Interview Q

URL-based vs method security; `hasRole` vs `hasAuthority` (ROLE_ prefix).

6.6 CSRF & CORS

CSRF (Cross-Site Request Forgery)

An attacker tricks a logged-in browser into making a state-changing request using its cookies. Defense: **synchronizer token** (Spring's `CsrfFilter`) or SameSite cookies. **Stateless JWT-in-header APIs are not vulnerable** to classic CSRF (no ambient cookie), so CSRF is commonly disabled for them — but *cookie-based* auth needs CSRF protection.

CORS (Cross-Origin Resource Sharing)

Browser same-origin policy blocks cross-origin JS calls unless the server sends `Access-Control-Allow-*` headers. Configure allowed origins/methods/headers; handle preflight `OPTIONS`. CORS is a *browser* mechanism, not a security boundary.

ASCII — CSRF vs CORS

```
CSRF: browser auto-sends cookies -> server must verify a token it can't forge
CORS: browser blocks cross-origin reads unless server opts in via headers
```

Interview Q / Follow-ups

- What is CSRF; why can you disable it for a stateless JWT API but not a cookie one?
- CORS preflight — when does it happen? *(non-simple requests: custom headers, PUT/DELETE, etc.)*
- Is CORS a security feature? *(browser-enforced; not server-side authorization.)*

Module 6 — One-Page Cheat Sheet

Topic	Key point
Filter chain	ordered filters before DispatcherServlet
AuthN flow	filter → AuthenticationManager → Provider → UserDetailsService + PasswordEncoder → SecurityContext
Context	SecurityContextHolder (ThreadLocal)
Passwords	BCrypt/Argon2, salted, DelegatingPasswordEncoder
Session vs JWT	stateful+revocable vs stateless+scalable
JWT	header.payload.signature; payload not encrypted; validate exp/iss/aud
OAuth2	code+PKCE (web), client-credentials (service); resource server validates JWT via JWKS
OIDC	adds ID token (identity) on top of OAuth2
Method security	@PreAuthorize SpEL
CSRF	needed for cookie auth; skip for header-token stateless
CORS	browser opt-in via Allow-* headers; not authz

Module 6 — Top Interview Questions

1. Walk through the security filter chain and authentication flow.
2. `AuthenticationManager` vs `Provider` vs `UserDetailsService`.
3. Session vs JWT; how to revoke a JWT.
4. OAuth2 authorization code + PKCE; OAuth2 vs OIDC.
5. Why BCrypt; how does salting work.
6. CSRF — what is it and when to enable/disable.
7. CORS vs CSRF; preflight requests.
8. URL vs method security; `@PreAuthorize`.
9. How to add a custom authentication filter.
10. How a resource server validates a JWT (JWKS).

Module 6 — Common Mistakes

- Disabling CSRF on a cookie-based app.
- Long-lived, non-revocable JWTs; secrets in payload.
- Treating CORS as authorization.
- Storing plaintext / fast-hashed passwords.
- Confusing 401 (unauthenticated) with 403 (unauthorized).

Module 6 — Mock Interview

1. *"Explain what happens from `POST /login` to a populated SecurityContext."* → UsernamePasswordAuthenticationFilter → AuthenticationManager → DaoAuthenticationProvider → UserDetailsService + PasswordEncoder → SecurityContext(+session).
2. *"We use JWTs; how do you log a user out immediately?"* → short TTL + refresh rotation + server-side denylist keyed by jti.
3. *"Frontend gets a CORS error."* → configure allowed origin/methods/headers + preflight; note it's browser-enforced.
4. *"Service-to-service auth in microservices?"* → OAuth2 client-credentials; each service a resource server validating JWT via JWKS.
5. *"401 vs 403?"* → 401 not authenticated (who are you); 403 authenticated but not permitted.

Next → Module 7: Microservices.